

Теория для лабораторной работы: Изучение работы веб-приложения на сервлетах и JSP

Эта лабораторная работа посвящена освоению основ разработки веб-приложений с использованием Java-сервлетов, JavaServer Pages (JSP) и интеграции с базой данных MySQL. Ниже представлена теоретическая база, которая объясняет ключевые концепции, используемые в работе, и поможет вам лучше понять процессы, происходящие в вашем проекте (веб-приложение меню ресторана для локальной сети) с использованием **Apache Tomcat 10.1.49**, **Java 23.0.1**, **MySQL**, **Apache NetBeans 24**, и **Windows 11**. Текущая дата и время — 08:10 PM EET, 15 ноября 2025.

1. Основы веб-приложений

Веб-приложение — это программное обеспечение, доступное через интернет или локальную сеть через браузер. Оно состоит из клиентской части (HTML, CSS, JavaScript) и серверной части (Java, PHP и др.). В вашем проекте серверная логика реализована на Java с использованием сервлетов и JSP, а клиентская часть формируется динамически на сервере.

- **Клиент-серверная архитектура:** Клиент (браузер) отправляет HTTP-запросы (GET, POST) на сервер, который обрабатывает их и возвращает ответ (HTML-страницу).
 - **Сервер приложений:** Apache Tomcat — это контейнер сервлетов, который управляет выполнением Java-кода и предоставляет доступ к веб-ресурсам.
-

2. Сервлеты (Servlets)

Сервлет — это Java-класс, расширяющий возможности сервера для обработки HTTP-запросов. Он является основным элементом серверной логики в Java веб-приложениях.

- **Определение:** Сервлет реализует интерфейс `jakarta.servlet.Servlet` и обычно наследуется от `HttpServlet`. В вашем проекте `MenuServlet` и `AddItemServlet` — примеры сервлетов.
- **Жизненный цикл:**
 1. **Инициализация:** Метод `init()` вызывается при загрузке сервлета Tomcat.
 2. **Обработка запросов:** Методы `doGet()` или `doPost()` обрабатывают HTTP-запросы.
 3. **Уничтожение:** Метод `destroy()` вызывается при остановке сервлета.
- **Аннотация `@WebServlet`:** Указывает URL-адрес (например, `/menu`), по которому сервлет доступен (например, `http://localhost:8081/RestaurantMenu/menu`).
- **Интеракция с JSP:** Сервлет передает данные в JSP через `request.setAttribute()` и перенаправляет с помощью `request.getRequestDispatcher()`.

Пример: В `MenuServlet` данные из базы загружаются и передаются в `menu.jsp` для отображения.

3. JavaServer Pages (JSP)

JSP — это технология, позволяющая встраивать Java-код и теги в HTML для создания динамических веб-страниц.

- **Структура:** JSP-файл (например, `menu.jsp`) содержит:
 - Статический контент (HTML, CSS).
 - Динамический контент (Java-код в виде скриплетов или JSTL-тегов).
- **Компиляция:** При первом запросе JSP компилируется в сервлет Tomcat, который генерирует HTML.
- **JSTL (JavaServer Pages Standard Tag Library):** Набор тегов (например, `<c:forEach>`) для упрощения работы с циклами и условиями. В вашем случае используется для перебора `menuItems`.
- **Директивы:** Например, `<%@page contentType="text/html" pageEncoding="UTF-8"%>` задает тип контента и кодировку.
- **Переменные запроса:** `${pageContext.request.contextPath}` дает доступ к контексту приложения (например, `/RestaurantMenu`).

Пример: В `menu.jsp` цикл `<c:forEach>` рендерит список блюд на основе данных от `MenuServlet`.

4. Интеграция с базой данных (MySQL)

Веб-приложение взаимодействует с базой данных для хранения и извлечения данных (например, меню ресторана).

- **JDBC (Java Database Connectivity):** API для подключения Java к базам данных. В вашем проекте используется драйвер MySQL (`mysql-connector-j-9.2.0.jar`).
- **Подключение:**
 - Класс `DatabaseConnection` создает `Connection` с помощью `DriverManager.getConnection()` .
 - URL: `jdbc:mysql://localhost:3306/restaurant?useSSL=false&serverTimezone=UTC` .
 - Учетные данные: `USER` и `PASSWORD` .
- **Запросы:** `PreparedStatement` обеспечивает безопасное выполнение SQL-запросов (например, `SELECT * FROM menu`).
- **Обработка ошибок:** Исключения `SQLException` логируются для отладки.

Пример: `MenuServlet` использует JDBC для загрузки данных из таблицы `menu` .

5. Стили и дизайн

Дизайн веб-приложений определяется HTML-структурой и CSS-стилями.

- **HTML:** Определяет структуру страницы (например, `<div>` , `<img>`).
- **CSS:** Управляет внешним видом (цвета, отступы, размеры). В вашем проекте используется Bootstrap и встроенные стили, а также можно создать `style.css` .
- **Bootstrap:** Фреймворк для адаптивного дизайна, подключаемый через CDN (`<link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">`).
- **Динамическая адаптация:** Путь к изображениям (`${pageContext.request.contextPath}${item.imagePath}`) обеспечивает корректное отображение статических файлов.

Пример: В `menu.jsp` классы Bootstrap (`container` , `row` , `col-md-4`) организуют компоновку.

6. Отладка и логирование

Отладка помогает находить и исправлять ошибки.

- **Логи:** Сообщения `System.out.println()` и `System.err.println()` выводятся в консоль NetBeans или файл `catalina.log` (например, `D:\Program\server\apache-tomcat-10.1.49\logs\catalina.2025-11-15.log`).
- **Исключения:** `SQLException` или `IOException` указывают на проблемы с базой или вводом-выводом.
- **Предупреждения Tomcat:** Например, `JDBC driver not unregistered` — это утечка памяти, которую можно игнорировать на этапе обучения.

Пример: В `MenuServlet` логи помогают отследить этапы загрузки данных.

7. Расширение функционала

Добавление новых функций (например, формы) расширяет возможности приложения.

- **HTTP POST:** Метод `doPost()` в `AddItemServlet` обрабатывает данные формы.
- **Перенаправление:** `response.sendRedirect()` возвращает пользователя на страницу меню после добавления.
- **Форма:** HTML-форма (`<form>`) отправляет данные (например, `name` и `price`) на сервлет.

Пример: `AddItemServlet` добавляет новое блюдо в базу и перенаправляет на `/menu`.

Ключевые понятия для понимания

- **Контекст приложения:** `/RestaurantMenu` — это корень вашего веб-приложения в Tomcat.
 - **Динамическая генерация:** JSP создает HTML на основе данных сервлета.
 - **Модульность:** Разделение логики (сервлеты) и представления (JSP) улучшает читаемость кода.
-

Использование в вашей лабораторной работе

- **Задание 1:** Проверяет базовую работу сервлета и JSP.
- **Задание 2:** Показывает создание нового сервлета и JSP.
- **Задание 3:** Обучает работе с CSS для дизайна.
- **Задание 4:** Развивает навыки отладки через логи.
- **Задание 5:** Практикует расширение функционала с формами.

Эта теория даст вам базу для выполнения заданий. Если что-то нужно уточнить, напишите — помогу! Удачи с лабораторной!

↳ [Расширь теорию о JSTL](#)

↳ [Сравни JSP с Thymeleaf](#)

↳ [Добавь примеры к теории](#)